

Coding & Solution

Date	29 September 2026
Team ID	SWTID-2026-7019
Project Name	EduGenie - AI Learning Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/mraguram1793-cmyk/Edugenie-Learning-assistant
Programming Language(s)	Python 3.13 (backend), JavaScript (front end), HTML5, CSS3
Framework(s) Used	FastAPI with Uvicorn, Jinja2 templates, Google Gen AI SDK (google-genai, Gemini), Hugging Face Transformers + PyTorch (local Flan-T5 model)
Key Features Implemented	• AI Q&A - answers any question (GET /qa) • Concept explanation in school-level language, local model with Gemini fallback (POST /explain/) • Text summarization in simple language (POST /summarize/) • Quiz generator - 3 multiple-choice questions returned as JSON (POST /quiz) • Interactive quiz UI with instant correct/wrong feedback and final score • Learning-path recommendations at beginner, intermediate and advanced level (GET /learn/recommendations) • Single-page web interface with loading states, toast messages and Enter-key support • Configuration via .env, API-key check, input validation and JSON error responses

Pending / Incomplete Features	• User accounts, login and saved progress or quiz history • Automated tests and a README • Pin dependency versions; home route needs the Starlette-1.x-compatible call <code>TemplateResponse(request, "index.html")</code> • Remove the hard-coded <code>D:\PyPackages</code> path and repeated Gemini client set-up; unify default model names • Return proper HTTP error codes for AI failures; add rate limiting and response streaming
Setup / Run Instructions	1. Install Python 3.10 or newer and clone the repository. 2. Create and activate a virtual environment: <code>python -m venv venv</code>, then <code>venv\Scripts\activate</code> (Windows) or <code>source venv/bin/activate</code> (macOS/Linux). 3. Install dependencies: <code>pip install -r requirements.txt</code> (transformers and torch are optional; without them explanations use Gemini). 4. Copy <code>.env.example</code> to <code>.env</code> and set <code>GEMINI_API_KEY</code> (optionally <code>GEMINI_MODEL</code> and <code>LOCAL_EXPLANATION_MODEL</code>). 5. Start the server: <code>uvicorn main:app --reload</code> 6. Open <code>http://127.0.0.1:8000</code> in a browser.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

Verified: the home page, static files, request validation (400 responses) and the missing-API-key guard respond correctly. AI responses require a valid `GEMINI_API_KEY` and were not exercised in this check. Compatibility: with current Starlette (1.x) the home route needs `TemplateResponse(request, "index.html")`; `requirements.txt` is unpinned, so pinning versions is recommended. Repository has 3 commits on the main branch.